

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Grafové databáze – přehled a srovnání



2024

Vedoucí práce:
Mgr. Tomáš Urbanec

Jan Gaja

Studijní program: Informatika,
Specializace: Obecná informatika

Bibliografické údaje

Autor:	Jan Gaja
Název práce:	Grafové databáze – přehled a srovnání
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2024
Studijní program:	Informatika, Specializace: Obecná informatika
Vedoucí práce:	Mgr. Tomáš Urbanec
Počet stran:	25
Přílohy:	1 CD/DVD
Jazyk práce:	český

Bibliographic info

Author:	Jan Gaja
Title:	Graph databases – overview and comparison
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2024
Study program:	Computer Science, Specialization: General Computer Science
Supervisor:	Mgr. Tomáš Urbanec
Page count:	25
Supplements:	1 CD/DVD
Thesis language:	Czech

Anotace

Práce má dva cíle: 1. nastudovat principy ukládání a práce s daty v grafových databázích a získané poznatky sepsat do textu, který čtenáři poskytne dobrou představu o fungování, výhodách a nevýhodách tohoto přístupu k datům. 2. vybrat několik implementací grafových databází, nachystat ukázky jejich použití a pokusit se navrhnout experimenty ukazující silné a slabé stránky zvolených implementací.

Synopsis

Thesis has two goals: 1. study the principles of storing and working with data in graph databases and and write down the knowledge gained in a text that will give the reader a good idea of how it works, advantages and disadvantages of this approach to data. 2. select several implementations of graph databases, prepare examples of their use and try to devise experiments showing the strengths and weaknesses of the selected implementation.

Klíčová slova: grafové databáze; implementace grafových databází

Keywords: graph databases; implementation of graph databases

Děkuji, děkuji, děkuji.

Místopřísežně prohlašuji, že jsem celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

datum odevzdání práce

podpis autora

Obsah

1	Úvod	7
2	Základní pojmy z teorie grafů	7
3	Modelování grafových dat	8
3.1	Definice problému	9
3.2	Konceptuální datový model	10
3.3	Logický datový model	11
3.4	Testování modelu	15
4	Algoritmy	16
4.1	Prohledávání grafu	16
4.2	Problém nejkratší cesty	17
4.3	Algoritmy pro míru centrality	17
4.3.1	Degree Centrality	17
4.3.2	Closeness Centrality	18
4.3.3	Betweenness Centrality	18
4.3.4	PageRank	18
4.3.5	Použití algoritmů pro míru centrality	18
4.4	Detekce komunit	19
4.4.1	Počítání trojúhelníků	19
4.4.2	Hledání komponent	21
4.5	Minimální kostra grafu	21
5	Implementace grafových databází	22
5.1	Neo4j	22
5.2	OrientDB	22
5.3	Dotazovací jazyky v grafových databázích	22
5.3.1	Gremlin	22
5.3.2	Cypher	22
5.3.3	GQL - Graph Query Language	22
	Závěr	23
	Conclusions	24
	Seznam zkratk	25

Seznam tabulek

1	Algoritmy pro měření centrality aplikované na jednotlivé vrcholy z grafu 6	19
2	Počet trojúhelníků a shlukovací koeficient jednotlivých vrcholů z grafu 6	20

1 Úvod

Grafové databáze, stejně jako jiné databáze (například relační, NoSQL) představují systém pro uchovávání a zpracovávání dat. Jsou, jak již název napovídá, postaveny na jednom z oborů diskrétní matematiky jménem teorie grafů, podobně jako jsou relační databáze postaveny na pojmu relace. Pracují tedy s daty v podobě grafu. Díky této struktuře mají skvělé vlastnosti pro aplikace, které ve větší míře pracují se vztahy mezi jednotlivými entitami.

2 Základní pojmy z teorie grafů

V této části jsou uvedeny základní definice z teorie grafů používané v textu.

Definice 1 (Graf)

Graf G je uspořádaná dvojice $\langle V, E \rangle$, kde V označuje množinu vrcholů a $E \subseteq V \times V$ označuje množinu hran. Každá hrana je určena dvěma vrcholy z V , které spojuje.

Grafy se dělí na orientované a neorientované. Hrana v neorientovaném grafu mezi vrcholy u a v se nazývá neorientovaná a zapisuje se pomocí množiny obsahující dvojici vrcholů, tedy $\{u, v\}$. V případě orientovaného grafu se hrana nazývá orientovaná a pro zápis se používá uspořádaná dvojice. Orientovaná hrana $\langle u, v \rangle$ začíná ve vrcholu u a končí ve vrcholu v . Grafy se nejčastěji znázorňují pomocí kroužků, které představují vrcholy, a úseček mezi nimi, sloužící pro reprezentaci hran. Pokud se jedná o orientovaný graf místo úseček používáme šipky. Pro hrany používáme označení násobné, pokud existuje mezi dvěma vrcholy hran více. O hraně začínající i končící ve stejném vrcholu mluvíme jako o smyčce.

Definice 2 (Podgraf)

Podgraf grafu $G = \langle V, E \rangle$ je graf $H = \langle V1, E1 \rangle$, kde $V1 \subseteq V$ a $E1 \subseteq E$, tak že každá hrana z $E1$ má oba vrcholy ve $V1$.

Definice 3 (Stupeň vrcholu)

Stupněm vrcholu u v grafu $G = \langle V, E \rangle$ se rozumí počet hran dotýkajících se u . Značíme jej $\deg(u)$. Stupeň vrcholu u lze v orientovaném grafu dále rozdělit podle hran vcházejících a vycházejících na vstupní $\deg_{in}(u) = |\{\langle u, v \rangle | v \in V\}|$ a výstupní $\deg_{out}(u) = |\{\langle v, u \rangle | v \in V\}|$.

Definice 4 (Sled)

Sled v grafu $G = \langle V, E \rangle$ je sekvence vrcholů a hran tvaru $v_0, e_1, v_1, e_2, \dots, e_n, v_n$, kde vrchol $v_i \in V$ pro všechna $i \in \{0, 1, 2, \dots, n\}$ a pro hranu e_i platí:

- $e_i = \{v_{i-1}, v_i\} \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G neorientovaný.
- $e_i = \langle v_{i-1}, v_i \rangle \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G orientovaný.

Definice 5 (Cesta)

Cesta je sled, kde se vrcholy ani hrany neopakují. Cesta délky n má $n + 1$ vrcholů.

Definice 6 (Vzdálenost)

Vzdálenost z vrcholu u do vrcholu v lze definovat jako délku nejkratší cesty z u do v . Značíme ji $dist(u, v)$.

Definice 7 (Kružnice)

Kružnice je cesta, kde počáteční a koncový vrchol je stejný. U orientovaného grafu používáme pojem cyklus.

Definice 8 (Souvislý graf)

Neorientovaný graf $G = \langle V, E \rangle$ nazýváme souvislý pokud pro každé dva vrcholy $u, v \in V$ platí, že existuje cesta z u do v . Komponenta grafu G je každý jeho maximální souvislý podgraf. U orientovaného grafu $H = \langle V_H, E_H \rangle$ rozlišujeme slabou a silnou souvislost. Graf H je slabě souvislý, pokud jeho symetrizace je souvislý graf. O grafu H řekneme, že je silně souvislý, pokud pro každé dva vrcholy $u, v \in V_H$ platí, že existuje cesta z u do v .

Symetrizace grafu lze popsat jako proces, ve kterém zapomeneme orientaci hran, odstraníme smyčky a vícenásobné hrany nahradíme jednoduchými a vznikne nám tak neorientovaný graf.

Definice 9 (Strom)

Strom je souvislý graf $G = \langle V, E \rangle$ neobsahující kružnice.

3 Modelování grafových dat

POZNÁMKA 10 (PŘEKLAD)

V práci je použito české slovo “vlastnosti” přeložené z anglického slova “properties”, se kterým se běžně uživatel setká při práci s grafovou databází. Slovo “label” je z důvodu znělosti zachováno v původním znění.

Modelování dat je proces, který překládá entity reálného světa a vztahy mezi nimi do jejich softwarové reprezentace. Uživatel popisuje doménu jako graf skládající se z vrcholů a hran pomocí vlastností a labelů. Každý výsledný model je unikátní. U každého záleží na případech použití a otázkách, které mají podobu dotazů, které budou na databázi kladeny. Dodržení základních principů modelování dat je důležité pro výslednou efektivitu. Když je graf přizpůsoben dotazům, dokáže velmi rychle nalézt potřebnou odpověď. Naopak špatně zvolený model grafu může ve výsledku aplikaci ublížit a být překážkou. Další důležitou otázkou je budoucnost vyvíjené aplikace. Model by měl být připravený na změny, které po aplikaci budou její koncoví uživatelé požadovat. Správnému modelu je proto

dobré věnovat dostatečné množství času a dodržovat obecná pravidla.

Postup modelování se dá rozdělit do čtyř fází:

1. Definice problému
2. Konceptuální datový model
3. Logický datový model
4. Testování modelu

V této sekci se budeme snažit dobře navrhnout grafový model dat. Zkusíme si to na příkladu sociální platformy pro nadšence cestování.

3.1 Definice problému

V první fázi bychom měli zvážit co všechno by měla aplikace umět. Výsledkem by měl být její, co nejpřesnější, slovní popis. Pomoci nám může těchto pár otázek.

Co od naší aplikace uživatel očekává? Aplikace je sociální síť, proto by měla jistě umožnit se propojit s ostatními uživateli aplikace. Uživatelům by na základě jejich přátel, oblíbených aktivit a hodnocení destinací měly přicházet tipy na výlet. Aplikace by měla umožnit zazanamenat, že někteří uživatelé podnikli cestu společně.

Jaké typy informací naše aplikace potřebuje uchovávat, aby měla danou funkcionalitu?

- Všechny základní informace o **uživateli**, aby se s nimi ostatní mohli spojit - jejich jméno, příjmení, unikátní identifikátor, oblíbené cestovatelské aktivity.
- Informace o jednotlivých **destinacích** - jejich lokace, název, unikátní identifikátor, cestovatelské aktivity v rámci destinace.
- Uživatelé je umožněno psát **hodnocení** na jednotlivé destinace.
- Chtěli bychom také umožnit **seskupení uživatelů**, když jedou do dané destinace společně.

V reálném scénáři by projekt pravděpodobně zahrnoval mnoho dalších atributů, ale pro tuto ukázkovou aplikaci to ponecháme omezené. Pro zjednodušení se vyhneme také souřadnicovým informacím a vystačíme si s informací v rámci jakého státu se daná destinace nachází.

Aplikace by tedy měla zacházet s pojmy jako jsou uživatel, destinace, hodnocení, aktivity. Jak spolu tyto pojmy vzájemně souvisí v rámci aplikace?

- Uživatel **sleduje** jiného uživatele.
- Uživatel **se zúčastnil** cesty **do** destinace.

- Uživateli se **líbí** cestovatelská aktivita.
- Uživatel **napsal** recenzi **o** cestě.
- Destinace **nabízí** cestovatelskou aktivitu.

Na jaké otázky potřebují uživatelé najít odpověď?

- Jaké všechny uživatele sleduji? A koho sledují oni? Které uživatele nesleduji ale sledují je uživatelé, které sleduji?
- Najdi 3 nejlépe hodnocené destinace, které jsem nenavštívil, nabízející konkrétní cestovní aktivity?
- Které destinace navštívil uživatel X. Kolikrát je navštívil?
- Najdi mi všechny uživatele v rámci skupiny, s kterou cestuje uživatel X do destinace Y.
- Najdi X destinací nabízející aktivity, které se uživateli líbí.
- Pro konkrétní destinaci vrať měsíc v roce v kterém je nejlépe hodnocená.
- V kterých destinacích a před kolika lety jsem byl v tento konkrétní den a měsíc?

V reálném scénáři by otázek bylo mnohem více.

3.2 Konceptuální datový model

Dalším krokem je vyvinout konceptuální model. Výstupem by měl být přehledný diagram popisující důmyslně promyšlené schéma naší aplikace z business pohledu. Vyvarujeme se v něm pro nevyvojáře zbytečným detailům a zaměříme se pouze na to, jak data různých typů spolu v rámci vyvíjené aplikace korelují. Při vyvíjení datového modelu lidé často kreslí na tabuli, jak jsou data v rámci projektu propojena s dalšími daty, proto se tato fáze v různých literaturách nazývá whiteboard model. Vyžaduje identifikování a seskupení všech entit a vztahů, které se v naší aplikaci vyskytují. Začneme entitami. Entity většinou představují objekty, jako například uživatel, účet, destinace, které jsou v databázi uloženy a propojeny vzájemnými vztahy. Je třeba si rozmyslet, které entity lze seskupit. Pokud například naše aplikace zahrnuje uživatele, zaměstnance a administrátora je užitečné tuto skupinu entit sloučit do jedné. Jedním z postupů jak z našich poznámek z minulé fáze vytáhnout entity, je dívat se po podstatných jménech.

V naší aplikaci můžeme rozpoznat 6 různých entit:

- Uživatel
- Cesta



Obrázek 1: Entity vyskytující se v aplikaci sociální síť pro cestovní nadšence

- Destinace
- Aktivita
- Stát
- Recenze

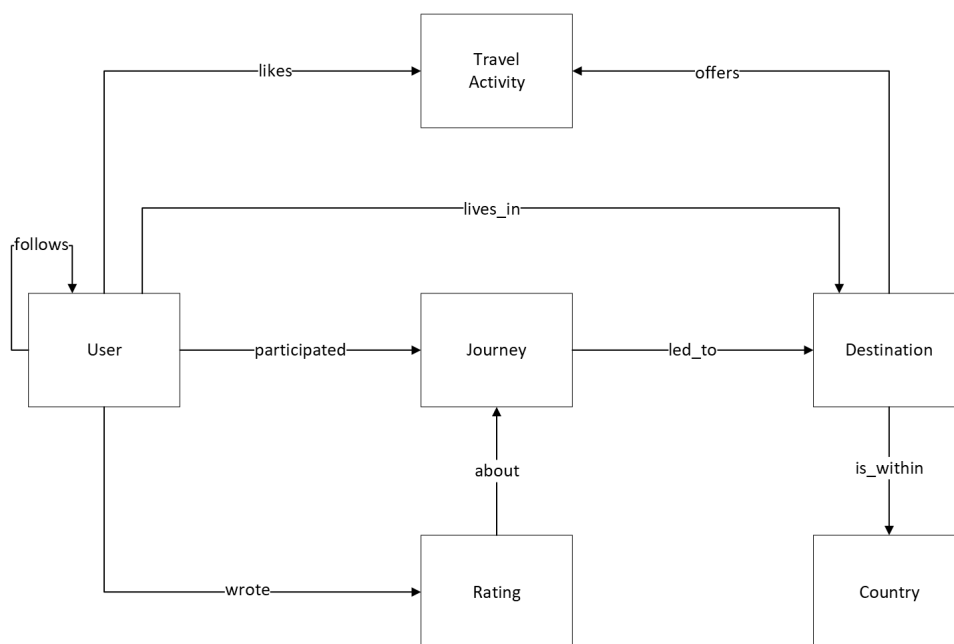
Naše tabule tedy má nyní podobu, kterou vystihuje Obrázek 1.

Nyní následuje propojení entit pomocí vztahů. Podobně jako entity lze zejména nalézt zaměřením se na podstatná jména, tak většina vztahů můžeme identifikovat pomocí sloves. Zde bychom však měli být velmi opatrní. Ve slovese se často ukrývá podstatné jméno. V budoucím vývoji aplikace by pak mohlo dojít k problémům při rozšiřování funkcionality aplikace. Dobrým příkladem je sloveso recenzovat. V případě, že bychom při modelování zvolili větu "Uživatel recenzoval X." místo "Uživatel napsal recenzi o X." mohli bychom narazit na problém, že nemáme entitu představující recenzi, na kterou bychom mohli odkazovat. Snadno si jde domyslet komentování nebo jiný způsob reagování na recenze. Podobně pokud bychom v naší aplikaci upřednostnili větu "Uživatel navštívil destinaci." před větou "Uživatel podnikl cestu do destinace.", nemohli bychom pak na danou cestu odkazovat. Takové chyby obecně vedou v budoucnosti k migrování databáze, což je velmi obtížný a zdlouhavý proces přesouvání dat z jedné databáze do druhé, kterému je lepší se vyhnout.

Po důkladném rozmyšlení naše tabule má obsah, jako Obrázek 2.

3.3 Logický datový model

Pokud jsme s diagramem z předchozí fáze spokojeni, můžeme konečně přistoupit ke kroku, ve kterém definujeme schéma, které bude v databázi. V případě relačního modelování je konceptuální model přeformátován, aby mohl být ukládán do



Obrázek 2: Konceptuální datový model pro sociální síť pro cestovní nadšence

tabulek. V případě grafového modelování toto není potřeba. Stačí identifikovat vrcholy a hrany a přiřadit jim příslušný label a vlastnosti. Výstupem tedy bude opět diagram, který ale bude mnohem detailnější. Bude se zabývat detaily, kterými se zabývají vývojáři. Bude definovat tvar, ve kterém budou data v databázi uložena a podle něj budeme moci klást na databázi dotazy.

Pro vytvoření grafového modelu jsou potřeba tyto 3 kroky:

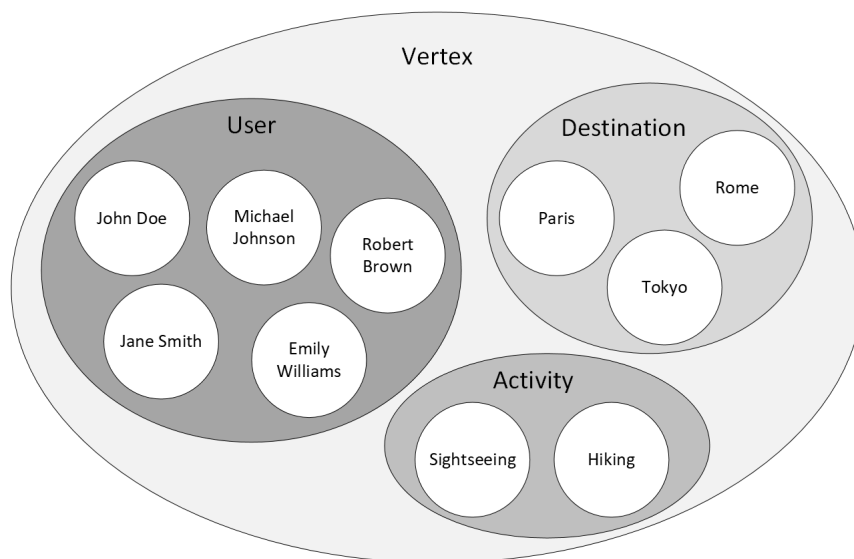
1. Převést entity na vrcholy
2. Převést relace na hrany
3. Najít a přiřadit vlastnosti vrcholům a hranám

První jsou na řadě vrcholy. Podstatnou část práce již máme za sebou, neboť můžeme využít konceptuální model a namapovat entity na vrcholy. Zbývá jen jeden netriviální úkol - každému vrcholu přiřadit krátký, přesný a výstižný label. Label vrcholu určuje, do které skupiny nebo kategorie vrcholů patří. Všechny vrcholy se stejným labelem pak spadají do stejné kategorie. Pro lepší pochopení je zde Obrázek 3. Label pak můžeme používat v dotazech na databázi a pracovat tak s různými skupinami uzlů a dosáhnout tak lepšího výkonu při dotazování.

Při vymýšlení labelu musíme mít jako vždy na paměti budoucí rozšiřitelnost a fakt, že s databází může pracovat jiný vývojář. Dobře zvolený label je tedy velmi žádaný. Důležité je též dbát na pojmenovovací konvenci.

POZNÁMKA 11 (KONVENCE POJMENOVÁNÍ)

V našem příkladu je zvolena konvence doporučená v dokumentaci databáze



Obrázek 3: Label vrcholu definuje do které kategorie vrchol spadá

Neo4j a tedy pro labely vrcholů budeme používat UpperCamelCase notaci, pro labely hran UPPER_CASE notaci s použitím podtržítka pro oddělení slov (SCREAMING_SNAKE_CASE) a pro vlastnosti camelCase notaci.

Dobrým zvykem je zvolit label, který bude v jednotném čísle, neboť každý uzel odkazuje na jednu instanci uzlu s daným labelem. Při přiřazování labelu je dobré myslet na jeho co největší obecnost, protože v budoucnu bychom mohli potřebovat reprezentovat jiný podobný koncept a tedy je užitečné volit co nejobecnější label.

Povolný počet labelů uzlu se u různých grafových databází liší. Bezpečným přístupem je každému vrcholu přiřadit jeden label. To je například přístup Apache TinkerPop. Často se ale také hodí mít labelů víc a pomocí nich umět vyjádřit dědičnost. Pokud bychom spravovali zaměstnance v nějaké firmě, hodilo by se někdy k labelu “Zaměstnanec” mít label “Ředitel”, “Manažer” apod. Tyto uzly pak spadají do více skupin uzlů naráz. Proto například Neo4j nebo Amazon Neptune umožňují uzlům přiřadit žádný, jeden ale i více labelů.

Při hledání hran opět využijeme důsledně připraveného konceptuálního modelu. Znovu se zamyslíme nad pojmenováním hran, které by opět mělo být obecné a výstižné. Navíc bychom se měli držet jedné dobré zásady. Přechod z vrcholu, hrany z něj vycházející a vrcholu do nějž hrana vstupuje by nám mělo dávat vcelku smysluplnou větu. V češtině však musíme opomenout skloňování. Tedy například “Uživatel napsal hodnocení.”, nebo “Uživatel sleduje uživatele.”. To nás přivádí k orientaci hran. Předchozí rada nám tedy specifikuje orientaci hrany, neboť obrácením hrany bychom dostali věty “Hodnocení napsalo uživatele.”, nebo “Uživatel sleduje uživatele.”. V druhém případě se nám smysl věty nezměnil, jelikož se jedná o smyčku jdoucí z jednoho vrcholu do toho samého

vrcholu. V prvním případě, kdy bychom požadovali opačnou orientaci hran mezi vrcholy “Uživatel” a “Hodnocení”, bychom museli přistoupit ke změně labelu hrany, neboť by věta postrádala smysl. Tedy například hraně “napsal” bychom změnili label na “bylo napsáno” a pak by věta “Hodnocení bylo napsáno uživatelem.” dávala smysl.

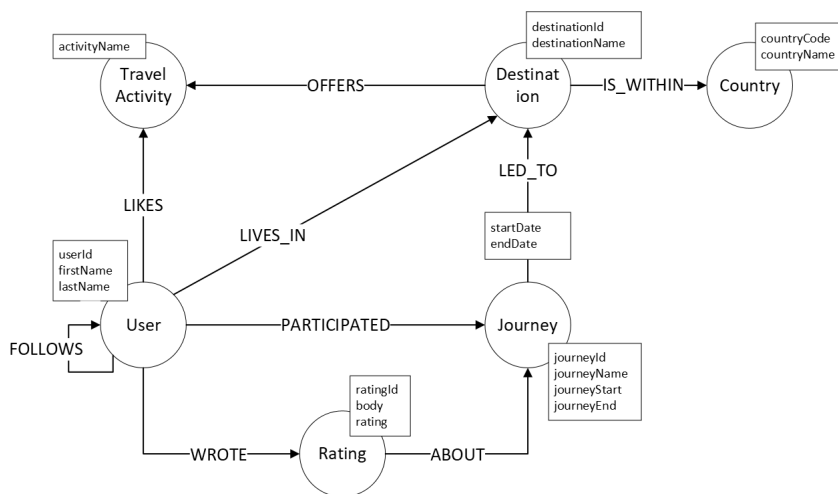
Než přejdeme k vlastnostem, zbývá ještě určit jedinečnost hran. Jedinečnost lze definovat jako počet hran se stejným labellem mezi dvěma vrcholy. Tyto vrcholy mohou a nemusí mít stejný label. Jedinečnost hran se dělí na dva typy jednoduchou a mnohonásobnou. Jednoduchá odkazuje na žádnou, nebo právě jednu hranu zatímco vícenásobná povoluje hran se stejným labellem mezi dvěma vrcholy více. Co se týká jedinečnosti hran v naší aplikaci tak všechny hrany, až na hranu mezi Cestou a Destinací budou mít jedinečnost jednoduchou. Mezi uživatelem a cestovní aktivitou vztah “LIKES” buď je nebo není a to lze vyjádřit pomocí existence jedné hrany mezi těmito dvěma uzly. Totéž platí pro ostatní hrany až na hranu s labellem “LED_TO”. Jistě totiž existuje situace, kdy v rámci cesty uživatel do dané destinace zavítá vícekrát a s touto možností by náš model měl také počítat. Tedy tato hrana bude mít vícenásobnou jedinečnost. Tyto hrany pak od sebe rozlišíme pomocí vlastností, které budou určovat období, kdy se daný uživatel v dané destinaci nacházel.

Jak zjistíme, že jsme zvolili špatnou jedinečnost hran? Špatná volba se projeví v dotazech jedním z těchto způsobů:

- Databáze vrací příliš málo dat. Tato chyba nastane pokud zvolíme jednoduchou jedinečnost hran tam, kde by měla být zvolena vícenásobná jedinečnost.
- Databáze vrací duplikativní data. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost.
- Slabý výkon dotazů na databázi. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost. Každá hrana stejného labelu mezi dvěma vrcholy navíc znamená rozvětvení v případě průchodu grafem a zvyšuje tedy výslednou časovou složitost dotazu.

U některých implementací grafových databází, jako například DataStax Graph nebo JanusGraph, lze tuto jedinečnost vynutit jako součást při definování schématu. U implementací bez schématického přístupu, jako je Neo4j, tuto jedinečnost lze ošetřit pouze v rámci aplikace.

Poslední zastávkou při konstruování logického modelu jsou vlastnosti. Základní kameny grafu vrcholy a hrany máme, ale chybí nám způsob jak rozlišit uzly se stejným labellem. K tomu slouží vlastnosti. V grafových databázích mohou mít vlastnosti jak hrany tak vrcholy, neboť hrany jsou pro nás stejně významné jak vrcholy. Vlastnosti v grafové databázi jsou páry klíč-hodnota a popisují konkrétní atributy hrany, či vrcholu. Hodnota vlastnosti může být různého datového



Obrázek 4: Logický datový model pro sociální síť pro cestovní nadšence

typu, například řetězec, číslo, pravdivostní hodnota nebo datový typ pro uchování datumu a času. Někdy též může být hodnota typu pole nebo seznam. Pomocí vlastností následně můžeme rozlišit jednotlivé uživatele pomocí jejich křestního jména, příjmení, uživatelské přezdívky apod. K dokončení logického modelu nám tedy zbývá najít a správně přiřadit vlastnosti konkrétním vrcholům a hranám, pojmenovat je a určit jejich datový typ. K jejich nalezení nám pomohou otázky, které budeme chtít, aby nám databáze zodpověděla. Z nich můžeme jednotlivé vlastnosti identifikovat a přiřadit.

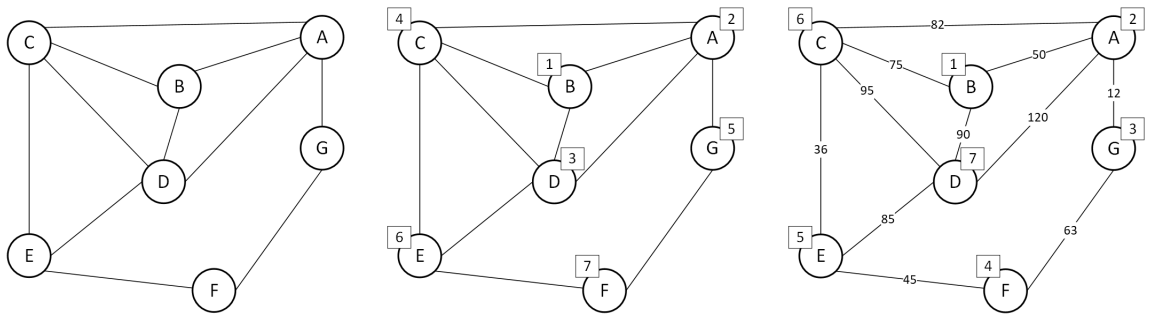
Náš výsledný datový model popisuje Obrázek 4.

3.4 Testování modelu

V budoucnu je možné přidat entitu cestovní kanceláře, která bude pořádat cesty do destinací a uživatelé po objednání budou do této cesty připojeni. To otevírá dveře dalším entitám například pro hodnocení cestovních kanceláří, nebo vztahům které se týká zaměstnanosti uživatelů u dané cestovky.

Testování našeho grafového modelu spočívá ve vyzkoušení, zda se chová dle našeho očekávání (databáze vrací očekávané výsledky) a dotazování není moc pomalé ani pro velké množství dat. K testování lze využít široké škály nástrojů, které dotazovací jazyky a databáze nabízejí.

Při testování dotazů je vhodné použít tzv. unit testy. Tedy vytvoříme si pro každý dotaz malé množství testovacích dat (například kolem 10 uzlů, tak abychom z něj pouhým okem vyčetli správnou odpověď), kterými otestujeme správnost odpovědí z databáze. Příklad unit testu je na Obrázku... TODO



Obrázek 5: Průchod algoritmů DFS(uprostřed) a BFS(vpravo) grafem(vlevo). Pořadí, v kterém jsou uzly procházeny je znázorněno čísla v obdélníčkách u každého uzlu.

4 Algoritmy

V této kapitole si ukážeme řadu užitečných algoritmů, které jsou určeny pro práci s grafem a jsou tedy v určitých situacích vhodné aplikovat při dotazech na grafovou databázi. Umožňují nám efektivně zjistit dosažitelnost vrcholů, hledat nejkratší cesty mezi vrcholy, nacházet uzly, které pro nás mohou být svým způsobem důležité, nebo třeba analyzovat komunity (skupinky propojených uzlů) v grafu.

4.1 Prohledávání grafu

Pro prohledávání grafů od určitého počátečního vrcholu lze použít dva základní algoritmy:

1. Prohledávání do šířky (zkráceně BFS z anglického spojení Breadth-first search)
2. Prohledávání do hloubky (zkráceně DFS z anglického spojení Depth-first search)

První zmíněný algoritmus začne v počátečním vrcholu a následně prohledá všechny své sousední vrcholy. Jakmile projde všechny, projde sousedy sousedů počátečního uzlu a tak postupuje dokud nenavštíví všechny uzly. Lze jej efektivně implementovat pomocí fronty vrcholů. Druhý algoritmus postupuje opačně, v každém uzlu přejde do dosud nenavštíveného souseda, pokud takový soused existuje, jinak provede backtracking - tedy vrátí se do předchozích vrcholů, než nenalezne vrchol jehož soused nebyl navštíven. Při implementaci využíváme zásobník. Oba algoritmy mají složitost $O(|E| + |V|)$. Průchod grafem pomocí obou algoritmů je znázorněn na Obrázku 5.

4.2 Problém nejkratší cesty

V praxi je často nezbytné hledání nejkratší cesty mezi dvěma, či více vrcholy. Tento problém potřebují řešit například aplikace pracující s mapami, kde se člověk potřebuje dostat z bodu A do bodu B. Cest mezi těmito body může být více. Jedna bude nejrychlejší, protože využívá vysokorychlostní silnice, tedy budeme brát v úvahu rychlosti na daných silnicích a celkovou vzdálenost. Jiná cesta může být nejkratší a tedy bereme cestu, která má z bodu A do bodu B nejkratší vzdálenost. Dalším příkladem může být hledání nejvhodnějšího autobusového spojení mezi dvěma zastávkami.

Nejkratší cesta mezi dvěma vrcholy v grafu s neohodnocenými hranami představuje cestu s nejmenším počtem hran. Počet hran na cestě tedy určuje délku cesty. Tento problém lze vyřešit pomocí algoritmů na prohledávání grafů. Konkrétně se řeší pomocí prohledávání do šířky, neboť systematicky prochází všechny vrcholy, které mají nejkratší vzdálenost od počátečního vrcholu 1, pak 2, 3 atd. dokud nenarazí na cílový vrchol.

ohodnocené hrany

4.3 Algoritmy pro míru centrality

V rámci grafu můžeme identifikovat důležitost vrcholů pomocí algoritmů centrality. Můžeme detekovat jedince, které jsou svým způsobem v grafu významější než ostatní. Tyto algoritmy se dají využít například k hledání kritických částí v telekomunikačních nebo počítačových sítích, tedy těch přes které proudí spousta informací a jejichž výpadek by představoval velké potíže. Hledání důležitých lidí v rámci organizace. Hledání odlehklých dat, které mohou odhalit, že dochází k podvodům a podobně. Knihovna gds nabízí spoustu algoritmů pro měření centrality. My se podíváme na pár z nich.

4.3.1 Degree Centrality

Nejznámějším a nejjednodušší mírou centrality je degree centrality. Každému uzlu v síti přiřazuje skóre podle stupně daného uzlu. V sociálních sítích mívají vrcholy i stupně v řádu desítek až milionů (jedná se o vrcholy, reprezentující účty známých osobností nebo organizací). Těmto uzlům říkáme super-uzly. Abychom prošli všechny sousedy super-uzlu potřebujeme značně více času, než v případě obyčejného uzlu. To se negativně projevuje v časové odezvě u dotazů pracujících s takovými uzly. Degree centrality nám umožňuje takové uzly odhalit. Vrcholy s větším stupněm také lze obecně považovat za důležitější, neboť přes ně může proudit více informací díky velkému počtu hran. Příkladem může být graf reprezentující železniční síť. Čím více máme z dané železniční stanice spojení, tím spíše se jedná o významější stanici.

4.3.2 Closeness Centrality

Vrcholy lze dále hodnotit podle closeness centrality. Vrchol s nejvyšším skóre je ten, pro který je součet nejkratších cest do všech ostatních uzlů co nejmenší. Skóre vrcholu $u \in V$ v grafu $G = \langle V, E \rangle$ se pak počítá podle vzorce

$$\text{closenessCentrality}(u) = \frac{1}{\sum_{v \in V} \text{dist}(u, v)}, \quad (1)$$

nebo se normalizuje

$$\text{normalizedClosenessCentrality}(u) = \frac{|V| - 1}{\sum_{v \in V} \text{dist}(u, v)}. \quad (2)$$

V případě, že graf není souvislý, lze použít harmonic centrality, které se elegantně vypořádá s nedosažitelnými uzly. Tento a následující algoritmus mohou být v případě velkých grafů výpočetně dost náročné a tedy se často přistupuje k ne zcela přesným, zato mnohonásobně rychlejším, algoritmům.

4.3.3 Betweenness Centrality

Algoritmus spočítá nejkratší cesty mezi všemi možnými páry vrcholů. Následně každý vrchol obdrží skóre, které je tím vyšší, čím více nejkratších cest vede přes daný vrchol. Tyto hodnoty mohou pro nás být klíčové například v grafu, kde uzly představují křižovatky a hrany cesty mezi nimi. Informaci, kterými křižovatkami prochází nejvíce nejkratších cest, můžeme využít pro odhad těch nejvíce zatížených křižovatek. Případně díky ní můžeme odhadnout, kde se pravděpodobně bude pohybovat nejvíce lidí a tedy je potřeba zvýšit bezpečnost.

4.3.4 PageRank

S tímto řešením přišla na trh společnost Google. Byla určena pro hodnocení významnosti webových stránek. Webové stránky si lze představit jako vrcholy grafu. Hrany mezi nimi reprezentují odkazy na stránkách odkazující na jiné stránky. Důležitost vrcholu je určena nejen důležitostí vrcholu samotného, ale také důležitostí vrcholů na tuto stránku odkazujících. Vysoké skóre vrcholu tedy znamená, že je vrchol propojen s vrcholy, které mají též vysoké skóre. Toto řešení je založeno na lineární algebře, speciálně na vlastních vektorech a stochastických maticích. Výpočet skóre většinou vede na nekonečný výpočet a proto se omezuje počet iterací tohoto výpočtu, nebo se udává maximální možná odchylka. PageRank se používá na orientované grafy. Pro neorientované grafy lze použít algoritmus eigenvector centrality.

4.3.5 Použití algoritmů pro míru centrality

Výsledky aplikace jednotlivých algoritmů pro měření centrality na graf z Obrázku 6 je znázorněn v tabulce 1.

Vrchol	A	B	C	D	E	F	G
degreeCentrality	4	3	4	4	3	2	2
closenessCentrality	0.75	0.6	0.75	0.75	0.67	0.55	0.6
betweennessCentrality	3.33	0.0	1.33	1.33	2.67	1.0	1.33
eigenVectorCentrality	0.45	0.41	0.49	0.49	0.32	0.14	0.17

Tabulka 1: Algoritmy pro měření centrality aplikované na jednotlivé vrcholy z grafu 6

4.4 Detekce komunit

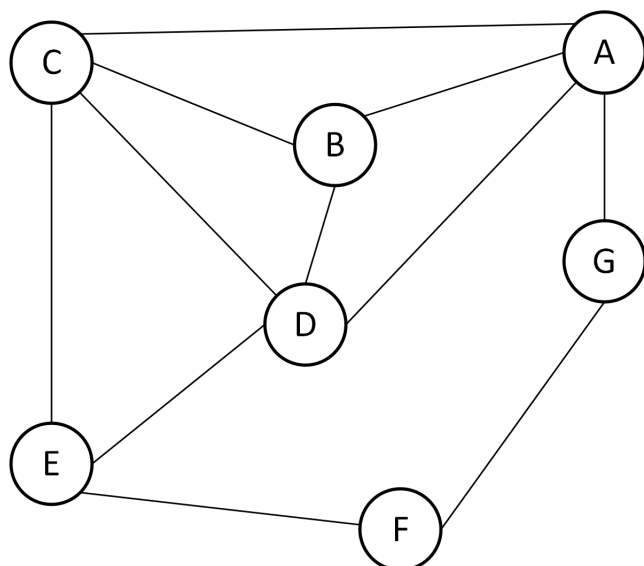
Tyto algoritmy lze využít pro hledání určitých skupin vrcholů, které mají mezi sebou více hran, než mají s ostatními vrcholy grafu. Odhalením těchto shluků vrcholů můžeme zkoumat strukturu našich dat z jiného úhlu pohledu, nebo třeba odhadovat podobné chování a preference členů v rámci skupiny. Příklad lze nalézt v sociálních sítích, kde můžeme sledovat komunity blízkých přátel, které tvoří shluky propojených vrcholů. Znalost komunit lze též využít pro doporučování produktů uživatelům s podobným nákupním chováním. Algoritmy pro detekci komunit se dělí na deterministické a nedeterministické. Deterministické vrací pro stejný graf vždy stejný výsledek, zatímco výsledek u nedeterministických může být pro stejný graf mírně odlišný. Mezi deterministické se řadí počítání trojúhelníků, výpočet shlukovacího koeficientu, nebo hledání komponent. Mezi nedeterministické patří například Label Propagation nebo Louvain Modularity. Stejně jako u míry centrality si zde ukážeme pouze pár algoritmů.

4.4.1 Počítání trojúhelníků

Algoritmus pro každý vrchol určí počet trojúhelníků, kterých je daný vrchol součástí. Trojúhelník je zde definován jako skupina tří navzájem propojených vrcholů. Trojúhelníky nám mohou určit do jaké míry jsou vrcholy v našem grafu propojeny. Čím je graf hustší, tím více trojúhelníků se v něm nachází. Algoritmus je oblíbený při analýze sociálních sítí, kde pomáhá detekovat komunity, nebo zkoumání skupin webových stránek s podobným námětem podle odkazů mezi nimi. Počet trojúhelníků lze využít též pro výpočet shlukovacího koeficientu vrcholu, který určuje pravděpodobnost, s jakou jsou jeho sousedé propojeni (tvoří s ním trojúhelník). Shlukovací koeficient vrcholu u je pak určen vztahem

$$ClusteringCoefficient(u) = \frac{2 * TriangleCount(u)}{deg(u) * (deg(u) - 1)}. \quad (3)$$

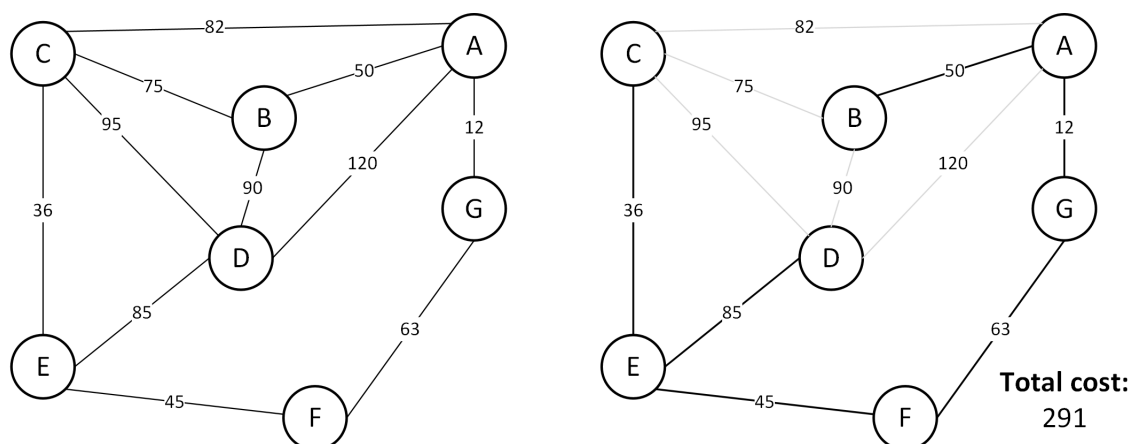
Pro ukázkový graf na Obrázku 6 počet trojúhelníků a shlukovací koeficient pro jednotlivé vrcholy zobrazuje tabulka 2. Vidíme, že vrchol A je součástí tří trojúhelníků, které jsou tvořeny vrcholy A-B-C, A-B-D a A-C-D. Stupeň vrcholu je 4. Dosazením do vzorce 3 dostáváme shlukovací koeficient 0.5.



Obrázek 6: Graf pro, který počítáme výskyt trojúhelníků

Vrchol	A	B	C	D	E	F	G
TriangleCount	3	3	4	4	1	0	0
ClusteringCoefficient	0.5	1.0	0.67	0.67	0.33	0.0	0.0

Tabulka 2: Počet trojúhelníků a shlukovací koeficient jednotlivých vrcholů z grafu
6



Obrázek 7: Minimální kostra grafu

4.4.2 Hledání komponent

Komponentu v grafu tvoří skupina uzlů, které jsou propojeny a lze se z jednoho uzlu dostat do dalšího. Najít těchto komponent je užitečné pro pochopení grafové struktury. Algoritmy řešící tento problém jsou rychlé a je možné je využít v grafu, který se často aktualizuje, pro detekci nových vrcholů mezi skupinami. Této znalosti lze využít při různých analýzách. Existuje i varianta, hledající silně souvislé komponenty. Pro graf na Obrázku 6 nám algoritmus `wcc` z knihovny `gds` vrátí, že všechny uzly grafu jsou součástí jedné komponenty.

4.5 Minimální kostra grafu

Dalším, mezi informatiky dobře známým, problémem je hledání minimální kostry v souvislém grafu. Vstupem je souvislý neorientovaný graf G s ohodnocenými hranami. Výstupem je minimální kostra grafu G . Kostru grafu lze definovat jako podgraf grafu G , který je stromem obsahujícím všechny vrcholy původního grafu. My požadujeme, aby kostra byla minimální, tedy aby součet ohodnocení hran v nalezeném podgrafu byl ze všech možných koster nejmenší. Příklad minimální kostry grafu je na Obrázku 7. Problém kostry grafu má více variant. Můžeme též hledat maximální kostru grafu, nebo omezit počet vrcholů v kostře na $k \in \mathbb{N}$ tomuto problému se říká k -spanning tree. Knihovna `gds` algoritmus na hledání kostry obsahuje. Konkrétně nabízí Jarníkův-Primův algoritmus, poprvé popsáný v roce 1930 českým matematikem Vojtěchem Jarníkem. Jedná se o hladový algoritmus, který vyžaduje na vstupu specifikovat i počáteční vrchol s . Vrátí minimální kostru komponenty vstupního grafu obsahující s . Pokud při implementaci využíváme například binární prioritní frontu máme celkovou složitost $O(|E| \log_2 |V|)$. Tento algoritmus lze využít pro nejlevnější propojení měst vodním potrubím, kde nižší ohodnocení hran znamená menší náklady. Dalším využitím je aproximace problémů patřících do třídy NPTIME, jako je například problém obchodního cestujícího.

5 Implementace grafových databází

Existuje spousta implementací grafových databází. Nejznámějšími jsou Neo4j, OrientDB, MemoGraph, TigerGraph, InfiniteGraph...

5.1 Neo4j

Jedná se o nejrozšířenější open-source grafovou databázi. Implementována je v Javě. Nabízí vysoký výkon při práci s grafy. Používá Cypher jako dotazovací jazyk.

5.2 OrientDB

OrientDB je open-source NoSQL databázový systém implementovaný v Javě. Je to multi-model databáze - podporuje grafový, dokumentový, hodnota/klíč a objektový model...

5.3 Dotazovací jazyky v grafových databázích

5.3.1 Gremlin

5.3.2 Cypher

5.3.3 GQL - Graph Query Language

Závěr

Závěr práce v „českém“ jazyce.

Conclusions

Thesis conclusions in “English”.

